

■ Krishi Bazar

Full-Stack MERN E-Commerce — Project Blueprint

Stack	MongoDB · Express · React · Node.js
Auth	JWT + bcrypt
Optional Features	Search · Category Filter · Pagination
Special Features	Rate Limiting · Request Logging · Response Time Tracking

This document contains the complete file structure and a flexible, phase-based development roadmap for your Krishi Bazar agricultural marketplace. Work at your own pace — each phase builds on the previous one.

■ SECTION 1 — Complete File & Folder Structure

Root Layout

```
krishi-bazar/  
  ■■■ backend/  
  ■■■ frontend/  
  ■■■ .gitignore  
  ■■■ README.md
```

Backend Structure (Node + Express + MongoDB)

```
backend/  
  ■■■ config/  
  ■ ■■■ db.js ← MongoDB connection  
  ■■■ controllers/  
  ■ ■■■ authController.js ← register / login logic  
  ■ ■■■ productController.js ← CRUD for products  
  ■ ■■■ cartController.js ← cart logic  
  ■ ■■■ orderController.js ← place & fetch orders  
  ■ ■■■ wishlistController.js ← (optional) wishlist  
  ■■■ middleware/  
  ■ ■■■ authMiddleware.js ← JWT verify  
  ■ ■■■ errorMiddleware.js ← global error handler  
  ■ ■■■ rateLimiter.js ← express-rate-limit  
  ■ ■■■ requestLogger.js ← morgan / custom logger  
  ■ ■■■ responseTime.js ← response-time tracking  
  ■■■ models/  
  ■ ■■■ User.js  
  ■ ■■■ Product.js  
  ■ ■■■ Cart.js  
  ■ ■■■ Order.js  
  ■ ■■■ Wishlist.js  
  ■■■ routes/  
  ■ ■■■ authRoutes.js  
  ■ ■■■ productRoutes.js  
  ■ ■■■ cartRoutes.js  
  ■ ■■■ orderRoutes.js  
  ■ ■■■ wishlistRoutes.js  
  ■■■ utils/  
  ■ ■■■ generateToken.js ← sign JWT  
  ■ ■■■ validators.js ← input validation helpers  
  ■■■ seed/  
  ■ ■■■ seedProducts.js ← dummy product data  
  ■■■ logs/  
  ■ ■■■ app.log ← request log file  
  ■■■ .env  
  ■■■ .env.example  
  ■■■ package.json  
  ■■■ server.js ← entry point
```

Frontend Structure (React + Vite / CRA)

```
frontend/  
  ■■■ public/  
  ■ ■■■ logo.png  
  ■■■ src/  
  ■ ■■■ api/  
  ■ ■ ■■■ axiosInstance.js ← base Axios config + JWT header  
  ■ ■■■ assets/ ← images, icons  
  ■ ■■■ components/  
  ■ ■ ■■■ Navbar.jsx
```

```
■ ■ ■ ■ Footer.jsx
■ ■ ■ ■ ProductCard.jsx
■ ■ ■ ■ CartItem.jsx
■ ■ ■ ■ Loader.jsx
■ ■ ■ ■ SearchBar.jsx
■ ■ ■ ■ CategoryFilter.jsx
■ ■ ■ ■ context/
■ ■ ■ ■ AuthContext.jsx ← user / token state
■ ■ ■ ■ CartContext.jsx ← cart state
■ ■ ■ ■ hooks/
■ ■ ■ ■ useAuth.js
■ ■ ■ ■ useCart.js
■ ■ ■ ■ pages/
■ ■ ■ ■ HomePage.jsx
■ ■ ■ ■ ProductPage.jsx
■ ■ ■ ■ CartPage.jsx
■ ■ ■ ■ CheckoutPage.jsx
■ ■ ■ ■ LoginPage.jsx
■ ■ ■ ■ RegisterPage.jsx
■ ■ ■ ■ NotFoundPage.jsx
■ ■ ■ ■ routes/
■ ■ ■ ■ PrivateRoute.jsx ← protects cart & checkout
■ ■ ■ ■ utils/
■ ■ ■ ■ helpers.js
■ ■ ■ ■ App.jsx
■ ■ ■ ■ main.jsx
■ ■ ■ ■ .env
■ ■ ■ ■ index.html
■ ■ ■ ■ package.json
```

■ *Keep backend and frontend in separate folders at the root level. This makes deployment to Render (backend) + Vercel (frontend) much easier.*

■ SECTION 2 — Environment Variables (.env)

backend/.env

```
PORT=5000
MONGO_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/krishibazar
JWT_SECRET=your_super_secret_key_here
JWT_EXPIRES_IN=7d
NODE_ENV=development
```

frontend/.env

```
VITE_API_URL=http://localhost:5000/api
```

SECTION 3 — Flexible Development Roadmap

This is phase-based — not day-based. Complete each phase fully before moving on. Each phase has clear goals and a checklist so you always know when you're done.

PHASE 1

Project Setup & Database

Get everything running locally

Goals

- Initialize both backend and frontend folders
- Connect to MongoDB Atlas (or local Mongo)
- Confirm the server starts without errors

Tasks — in order

#	Task	File(s)
1	mkdir krishi-bazar && cd krishi-bazar	—
2	cd backend && npm init -y	package.json
3	npm install express mongoose dotenv cors bcryptjs jsonwebtoken express-rate-limit morgan response-time express-validator	package.json
4	Create server.js with basic Express app	server.js
5	Create config/db.js — connect Mongoose	config/db.js
6	Add .env with MONGO_URI and PORT	.env
7	cd frontend && npm create vite@latest . --template react	—
8	npm install axios react-router-dom	package.json
9	Test: run both servers (nodemon + vite)	—

■ Use nodemon for auto-restart: `npm install -D nodemon` and add "dev": "nodemon server.js" to scripts.

PHASE 2

Models & Auth

Build User model + JWT Login/Register

#	Task	File(s)
1	Create User model (name, email, password, role)	models/User.js
2	Create authController — register with bcrypt hash	controllers/authController.js
3	Create authController — login, compare hash, sign JWT	controllers/authController.js
4	Create generateToken utility	utils/generateToken.js
5	Create authMiddleware — verify JWT on protected routes	middleware/authMiddleware.js
6	Create authRoutes — POST /api/auth/register & /login	routes/authRoutes.js
7	Mount routes in server.js	server.js
8	Test with Postman / Thunder Client	—

PHASE 3

Product CRUD & Seed Data

Core product API + dummy data

#	Task	File(s)
1	Create Product model (name, description, price, category, image, stock, seller)	models/Product.js
2	Create productController — GET all (with search, category filter, pagination)	controllers/productController.js
3	Create productController — GET single, POST, PUT, DELETE	controllers/productController.js
4	Create productRoutes — protect POST/PUT/DELETE with authentication	routes/productRoutes.js
5	Create seed/seedProducts.js — 15–20 agri products	seed/seedProducts.js
6	Run seed: node seed/seedProducts.js	—
7	Test all endpoints in Postman	—

PHASE 4	Cart & Order System	Add to cart, checkout, save orders
--------------------	--------------------------------	------------------------------------

#	Task	File(s)
1	Create Cart model (user ref, items array with product & qty)	models/Cart.js
2	cartController — GET cart, POST add item, PUT update qty, DELETE remove item	controllers/cartController.js
3	Create cartRoutes — ALL protected	routes/cartRoutes.js
4	Create Order model (user, items, totalPrice, status, address)	models/Order.js
5	orderController — POST place order (clear cart after), GET user orders	controllers/orderController.js
6	Create orderRoutes	routes/orderRoutes.js
7	Test full flow in Postman: login → add to cart → checkout	—

PHASE 5	Special Features (Backend)	Logging · Rate Limiting · Response Time
--------------------	-----------------------------------	---

#	Task	File(s)
1	Add morgan logger — log every request to console & logs/app.log	middleware/requestLogger.js
2	Add express-rate-limit — 100 req/15min on /api/*	middleware/rateLimiter.js
3	Add response-time middleware — X-Response-Time header on all responses	middleware/responseTime.js
4	Add global error handler middleware	middleware/errorMiddleware.js
5	Wire all middleware in server.js (order matters!)	server.js
6	Add input validation on register & product create with express-validator	validators.js

■ *server.js middleware order: responseTime → requestLogger → rateLimiter → cors → express.json() → routes → errorMiddleware*

PHASE 6	Frontend Foundation	Routing, Auth context, Axios setup
-------------------	----------------------------	------------------------------------

#	Task	File(s)
1	Set up React Router in App.jsx — define all page routes	App.jsx
2	Create AuthContext — store token & user, provide login/logout	context/AuthContext.jsx
3	Create CartContext — store cart items, counts	context/CartContext.jsx
4	Create axiosInstance with baseURL from .env + attach JWT header	utils/axiosInstance.js
5	Create PrivateRoute component — redirect to /login if no token	routes/PrivateRoute.jsx
6	Build Navbar with links + cart icon + auth state	components/Navbar.jsx

PHASE 7	Frontend Pages	Build all 5 core pages
-------------------	-----------------------	------------------------

#	Task	File(s)
1	LoginPage & RegisterPage — forms, call auth API, save tokens	pages/LoginPage.jsx pages/RegisterPage.jsx
2	HomePage — fetch & display all products, SearchBar, CategoryFilter	pages/HomePage.jsx
3	ProductCard component — image, name, price, Add to Cart button	components/ProductCard.jsx
4	ProductPage — fetch single product by ID, show details	pages/ProductPage.jsx
5	CartPage (protected) — list cart items, update qty, remove	pages/CartPage.jsx
6	CartItem component	components/CartItem.jsx
7	CheckoutPage (protected) — address form, submit order	pages/CheckoutPage.jsx
8	NotFoundPage — 404 fallback	pages/NotFoundPage.jsx

PHASE 8	Optional Features	Choose 2–3 from this phase
-------------------	--------------------------	----------------------------

Feature	Backend	Frontend
Product Search	Query param ?search= in GET /products	SearchBar component on HomePage
Category Filter	Query param ?category= in GET /products	CategoryFilter dropdown on HomePage
Pagination	?page=&limit= + total count in response	Prev/Next buttons on HomePage
Wishlist	Wishlist model + CRUD routes	Heart icon on ProductCard, WishlistPage
Recently Viewed	Store in localStorage	Show last 5 visited products on HomePage

■ *Recommended pick: Search + Category Filter + Pagination — they work together and are most impressive to reviewers.*

PHASE 9	Polish & Final Touches	README, testing, cleanup
-------------------	-----------------------------------	--------------------------

#	Task
1	Write README.md — project description, features, setup steps, env vars, API endpoints table
2	Add Loader component — show spinner while API calls are in progress
3	Add toast notifications for success/error (react-hot-toast or react-toastify)
4	Test all protected routes — make sure unauthenticated users are redirected
5	Test edge cases — empty cart, invalid product ID, wrong password
6	Clean up console.logs and unused imports
7	Confirm seed data works: node seed/seedProducts.js

PHASE 10	Deployment	Ship it live!
--------------------	-------------------	---------------

What	Where	Key Steps
Backend (Node/Express)	Render.com (free tier)	1. Push to GitHub 2. New Web Service on Render 3. Set env vars (MONGO_URI, JWT_SECRET) 4. Build cmd: npm install Start cmd: node server.js
Frontend (React/Vite)	Vercel (free tier)	1. Push to GitHub 2. Import project on Vercel 3. Set VITE_API_URL = your Render URL 4. Deploy
Database (MongoDB)	MongoDB Atlas (free M0)	1. Create cluster 2. Whitelist 0.0.0.0/0 (all IPs) 3. Copy connection string to MONGO_URI

■ After deploying, update your frontend VITE_API_URL to the Render production URL, then re-deploy Vercel. Also add your Vercel domain to MongoDB Atlas IP whitelist.

■ SECTION 4 — Quick Reference Cheatsheet

Key npm Packages

Package	Purpose	Side
express	Web framework	Backend
mongoose	MongoDB ODM	Backend
bcryptjs	Password hashing	Backend
jsonwebtoken	JWT sign/verify	Backend
express-rate-limit	Rate limiting / anti-spam	Backend
morgan	HTTP request logger	Backend
response-time	X-Response-Time header	Backend
express-validator	Input validation	Backend
dotenv	Load .env variables	Backend
cors	Allow cross-origin requests	Backend
axios	HTTP client for API calls	Frontend
react-router-dom	Client-side routing	Frontend
react-hot-toast	Toast notifications	Frontend

API Endpoints Summary

Method	Endpoint	Auth?	Description
POST	/api/auth/register	No	Register user
POST	/api/auth/login	No	Login, returns JWT
GET	/api/products	No	List products (search, filter, page)
GET	/api/products/:id	No	Single product
POST	/api/products	Admin	Create product
PUT	/api/products/:id	Admin	Update product
DELETE	/api/products/:id	Admin	Delete product
GET	/api/cart	Yes	Get user cart
POST	/api/cart	Yes	Add item to cart
PUT	/api/cart/:itemId	Yes	Update quantity
DELETE	/api/cart/:itemId	Yes	Remove item
POST	/api/orders	Yes	Place order (checkout)
GET	/api/orders	Yes	Get user orders

Middleware Order in server.js

```
app.use(responseTime())
app.use(requestLogger) // morgan
app.use(rateLimiter) // express-rate-limit
app.use(cors())
app.use(express.json())
app.use('/api/auth', authRoutes)
app.use('/api/products', productRoutes)
app.use('/api/cart', cartRoutes)
```

```
app.use('/api/orders', orderRoutes)
app.use(errorMiddleware) // LAST – catches all errors
```

Phase Progress Tracker (print & tick off!)

✓	Phase	Status
☐	Phase 1 — Project Setup & DB	
☐	Phase 2 — Models & Auth	
☐	Phase 3 — Product CRUD & Seed	
☐	Phase 4 — Cart & Orders	
☐	Phase 5 — Special Features	
☐	Phase 6 — Frontend Foundation	
☐	Phase 7 — Frontend Pages	
☐	Phase 8 — Optional Features	
☐	Phase 9 — Polish & Cleanup	
☐	Phase 10 — Deployment	

■ Good luck with Krishi Bazar! Build phase by phase, test each before moving on, and you'll have a solid full-stack MERN portfolio project.